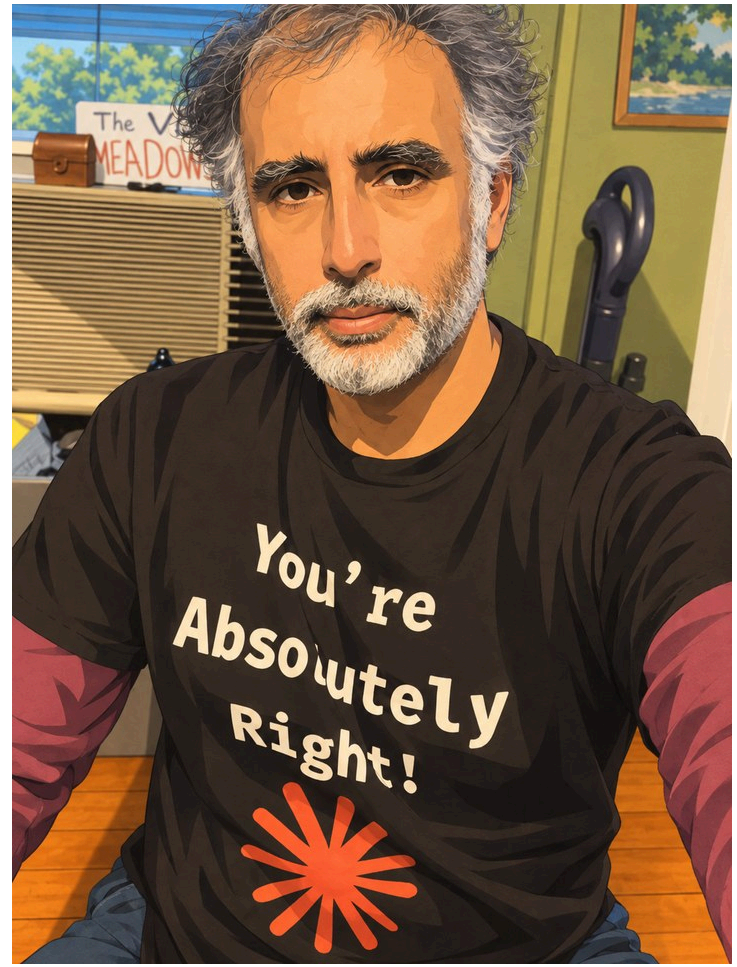
SOFTWARE DEVELOPMENT

FOR

MACHINE LEARNING

Rahul Dave



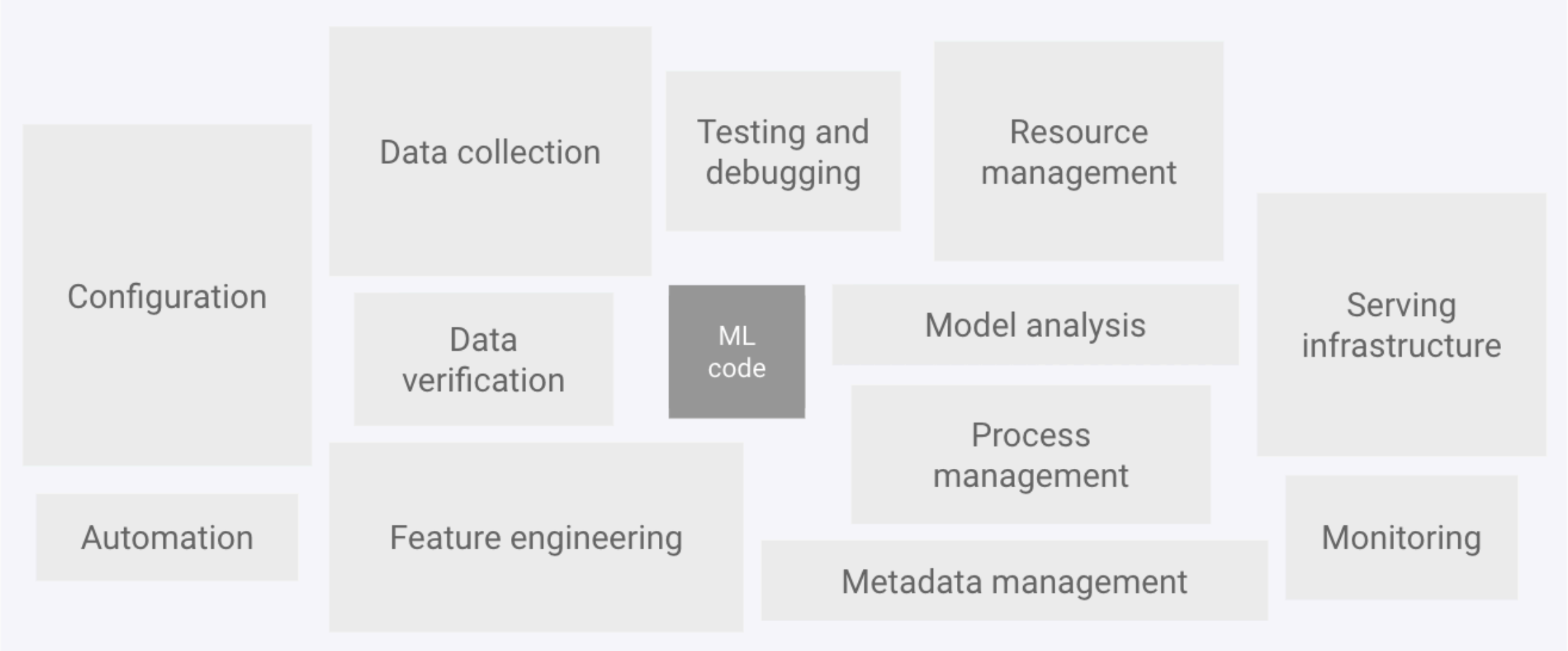

You're
Absolutely
Right!

What goes into a machine-learning project?

Design	Setup, Product, Systems
Data	Preparation, Exploration, Preprocessing, Distributed
Model	Training, Tracking, Tuning, Evaluation, Serving
Developing	Scripting, CLI
Utilities	Logging, Documentation, Styling, Pre-commit
Testing	Code, Data, Models
Reproducibility	Versioning
Production	Jobs & Services, CI/CD workflows, Monitoring, Data engineering

9/26 lessons aspects are model and data. The rest is engineering.

The ML code is a small box



Google Cloud, *MLOps: continuous delivery and automation pipelines in machine learning*, after Sculley et al., *Hidden technical debt in machine learning systems*, NeurIPS 2015

What goes into writing software?

Source	Version control, Code review
Environment	One known Python, every dependency pinned, one way to run anything
Code	A package, a command-line interface, types, docstrings
Quality	Formatting, linting, type checking
Tests	Unit tests, property tests, integration tests, system tests, coverage
Documentation	Reference built from code, guides, a site
Automation	Hooks on dev, continuous integration, builds
Shipping	Containers, packaging for others, releases, deployment
Running	Logging, monitoring, someone on call

A machine-learning project is a software project with model and data in it.

Today

- A skeleton project, a justfile, one environment from one lockfile. **uv**, **just**
- The contract for the model, `api.py`, five sample requests, and the tests. **pydantic**, **pytest**, **pytest-cov**
- A formatter and a linter **ruff**
- `just check` with a coverage floor, additional linters for the config files, and a git hook. **pre-commit**, **yamllint**, **validate-pyproject**
- `babykev docs`: the code's own documentation, as Markdown and as a site. Then every function documented. **docments**, **Quarto** . For humans and agents.
- The model, `model.py`; a smoke run on the real model; a tiny model for the tests; a type checker. **torch**, **transformers**, **peft**, **hypothesis**, **ty**
- The mask's rule as property tests, the bug they find, the one-line fix, ty in the gate, two guide pages
- The dev image and the run image from one Dockerfile, a linter for it, and the first end-to-end test in pytest: the smoke on the real model, kept out of `just test`. The site published. **Docker**, **hadolint**

babykev

A small model that makes decisions, and the project built around it.

- the code is taken from `kev`, a decider model with a special head to make it work like `jev`.
- We start from kev's first commit: about 830 lines that work, with no tests, no types and no docs
- We rebuild it in steps, a few files at a time
- At each step a new tool arrives, we learn about it and run it
- we incorporate the tool into a uniform interface, in the `Justfile`

There is a second tutorial example which uses docker through and through, which will be released later today.

The second example: the cheese app



github.com/dlops-io/cheese-app-ci-cd

A FastAPI app in Docker, with linters, formatters, tests and pre-commit hooks.

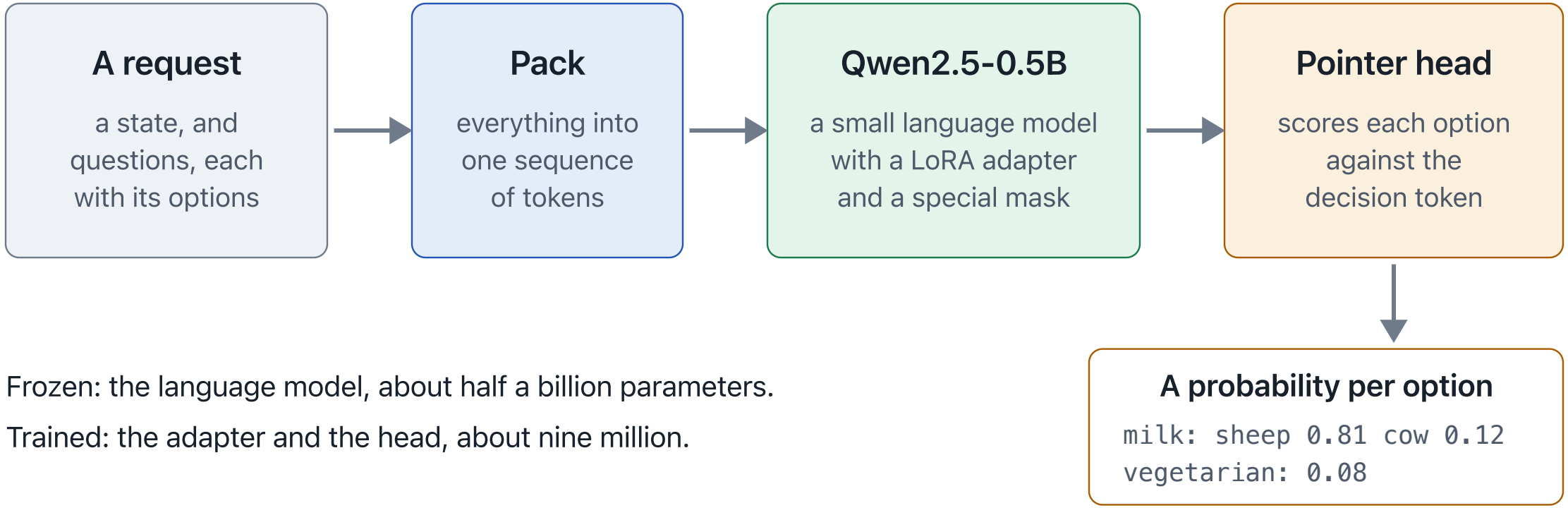
One request, one answer

```
state: "Roquefort is a semi-soft, blue-veined cheese from France. It tastes sharp, tangy and salty."
questions:
  milk:       {type: choice, criteria: {cow: …, goat: …, sheep: …, buffalo: …, mixed: …}}
  vegetarian: {type: noul}      # yes or no
  intensity:  {type: score, criteria: [mild, medium, pronounced, strong]}
```

```
milk:       {choice: sheep, confidence: 0.81}
vegetarian: {noul: 0.08}
intensity:  {score: 2.7}
```

Every answer is a probability. No text is generated.

A model that decides



Frozen: the language model, about half a billion parameters.

Trained: the adapter and the head, about nine million.

Questions are not allowed to see each other.

You install three tools. The project installs the rest

Tool	For
uv	Python itself, every dependency, running the program
just	The project's tasks: `just` lists them
Quarto	The documentation site, from step 04

Everything else (pydantic, pytest, ruff, pre-commit, torch, ty, hadolint, etc) comes from the lockfile, through `just setup`.

Where we start: kev's files, one at a time

File in kev's first commit	Lines	What it does	Arrives at
`api.py`	111	The contract: what a request and an answer look like	step 01
`model.py`	101	Packing, the mask, the pointer head	step 05
`data.py`, `train.py`, `evaluate.py`	446	Training records, the loop, accuracy and calibration	class on training
`serve.py`	170	Inference	class on ci/cd with github actions

uv: one tool for Python, environments and packages

Before	With uv
pyenv, to install a Python	nothing: uv fetches the one `.python-version` names
venv and pip, to make an environment	`uv sync`
pip-tools or poetry, to pin versions	`uv lock`, which writes `uv.lock`
pipx, to run a tool once	`uvx ruff`
"activate the environment first"	`uv run <command>`: no activation

Two files describe the environment

- `pyproject.toml` is written by people: what the project needs, loosely
- `uv.lock` is written by uv: exactly which version of everything, for every platform
- Both are committed. Every machine gets the same environment
- `uv sync --locked` installs exactly what `uv.lock` says. Say a new package needs a newer version of one you already have: it will not quietly upgrade it. It stops with an error, and the upgrade happens only when someone runs `uv lock` on purpose, which updates `uv.lock`
- The justfile sets `UV_LOCKED=1`, so every recipe behaves the same way

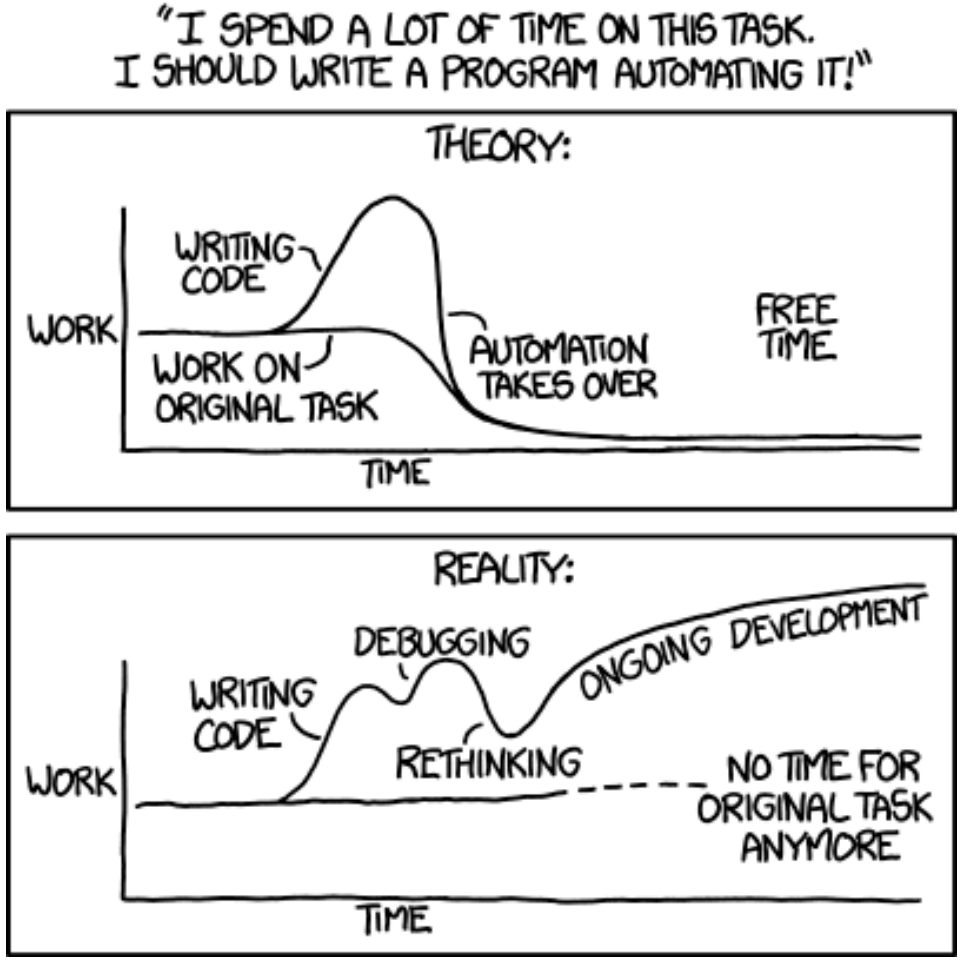
just: the project's interface

```
# Install Python and every dependency into .venv, exactly as the lockfile says
setup:
    uv sync --locked
```

- A recipe is a named command. `just setup` runs it
- `just` alone lists every recipe, with the comment above it
- `just --show setup` prints what is behind the name

Whoever calls a recipe does not need to know what is behind it. When the command changes, the call does not.

Why automate



xkcd.com/1319, *Automation*. CC BY-NC 2.5

The justfile, line by line

```
# babykev: every task is a recipe here. `just` lists them; `just help VERB` explains one.

program := "babykev"

# Each word after a recipe's name reaches its commands whole, as "$1", "$2" and so on; "$@" is all of them
set positional-arguments

[private]
default:
    @just --list --unsorted

# Explain a verb: its line, and the program's flags where the program has them. Alone, the list
[no-exit-message]
help *verb:
    #!/usr/bin/env sh
    if [ "$#" -eq 0 ]; then just --list --unsorted; exit 0; fi
    shown=$(just --show "$1") || exit 1
    printf '%s\n' "$shown" | head -1 | sed 's/^# //'
    if printf '%s\n' "$shown" | grep -qF '{{ "{{ program }}" }} '"$1"; then
        uv run {{ program }} "$1" help
    fi

# Install Python and every dependency into .venv, exactly as the lockfile says
[group('setup')]
setup *args:
    @{{ if args == "help" { "just help setup" } else { "uv sync --locked" } }}
```

A comment, for whoever opens the file

A variable. Below, {{ program }} becomes babykev

A setting for every recipe: just gives each word after a recipe's name to its commands whole, as "$1", "$2", and all of them as "$@"

[private]: leave this recipe out of the list.
default: the first recipe, run by `just` alone.
@: run the command without printing it first

The comment right above a recipe is its description: `just` prints it in the list

[no-exit-message]: on failure, just adds no line of its own

A recipe named help. *verb takes any number of words: `just help setup`

#! makes the body one shell script, not a command per line. It prints the recipe's description, then the program's own help when the recipe runs the program with its name

[group('setup')]: a heading in the list

*args: any words. {{ if … }} is just's own if:
`just setup help` explains the recipe,
`just setup` runs uv sync --locked

The help recipe, line by line

```
[no-exit-message]

help *verb:

    #!/usr/bin/env sh

    if [ "$#" -eq 0 ]; then just --list --unsorted; exit 0; fi

    shown=$(just --show "$1") || exit 1

    printf '%s\n' "$shown" | head -1 | sed 's/^# //'

    if printf '%s\n' "$shown" | grep -qF '{{ "{{ program }}" }} '"$1"; then

        uv run {{ program }} "$1" help

    fi
```

When a recipe fails, just adds a line of its own: `error: Recipe help failed with exit code 1`. This turns that line off: the script's message is enough

help: the recipe. *verb: any number of words, or none. It uses the first, `"$1"`, and ignores the rest

#!: the body is one sh script. Without it, just runs each line as a separate command

`$#` is how many words there are. None, as in `just help`: print the list, and stop

Keep the text of the recipe named by the first word, as `just --show` prints it. No such recipe: just says so, and we stop

Print its first line, the comment above it, without the `#` . That is the recipe's description

Does the recipe run the program with this word? This looks for the text {{ program }} and the word, as in {{ program }} docs. `-F`: plain text, not a pattern. {{ "..." }} writes the braces as they are

If it does, ask the program for its own help too, as `uv run babykev docs help` from step 04. At step 00 no recipe runs the program with its own name, so this never happens yet

```
$ just help setup
Install Python and every dependency into .venv, exactly as the lockfile says

$ just help nosuch
error: Justfile does not contain recipe `nosuch`
```

How to ask what a recipe does

```
just                 the list, grouped
just help test       one recipe: its line, and the program's own help where there is one
just test help       the same, the other way round
just --show test     the recipe itself
```

- Extra words go to the tool behind the recipe: `just test -k render` passes `-k render` to pytest
- People, agents, the git hook and CI all run the same recipes

`api.py` is the contract

It says what a request is and what an answer is, as Pydantic models.

- `SystemOneRequest`: a state, and questions by name
- `render`: turn any JSON value into the text the model reads
- `to_record`: turn a request into the record the model is given
- `to_answers`: turn the model's probabilities back into answers

Three kinds of question, one mechanism

Kind	You give	You get back
`noul`	a yes-or-no question	the probability of yes
`choice`	named options, each with a description	the most probable option, a confidence, every probability
`score`	ordered levels	the expected level, a confidence, every probability

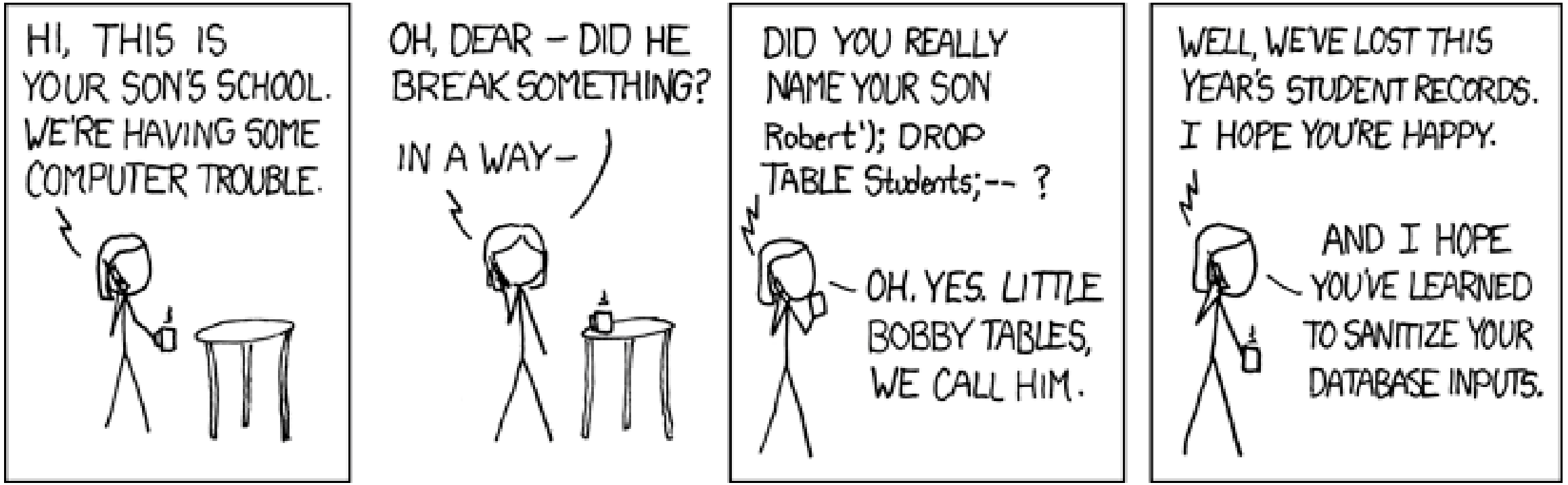
Each becomes a list of options. The model only ever points at one option in a list.

Pydantic: types checked when the data arrives

```
class Score(BaseModel):
    type: Literal["score"]
    instructions: JSONContent
    criteria: list[JSONContent] = Field(min_length=2, max_length=MAX_OPTIONS)
```

- A **named** class says what the data must look like. Pydantic validates on this.
- Validation happens before the code runs.
- Look at sample data in `data/cheese/sample.jsonl`.

Check what comes in



xkcd.com/327, *Exploits of a Mom*. CC BY-NC 2.5

pytest: a test is a function that asserts

Kind: unit: one function, alone. Every test at this step is a unit test; the other kinds come with the model.

```python
def test_choice_confidence_is_zero_for_a_guess() -> None:
    assert choice_confidence([0.25, 0.25, 0.25, 0.25]) == 0.0
```

- `tests/unit/test_*.py`. `just test` runs them all, `just test -k confidence` some
- Every run measures coverage: which lines of `src/babykev` the tests reached
- At step 01: 52 runs, 50 pass, **2 fail**. Coverage 100%
- What to test: normal inputs, edge cases, invalid inputs. Both failing tests are edge cases, a pure guess and 77 options. pydantic refuses invalid inputs

Testing the command: fixtures and parametrize

Kind: unit: `main`, alone, with the words it would get from a terminal.

```
def run(monkeypatch, *words):
    monkeypatch.setattr("sys.argv", ["babykev", *words])
    main()

@pytest.mark.parametrize("words", [(), ("help",), ("--help",), ("-h",)])
def test_help_is_printed_for_help_and_for_no_words(monkeypatch, capsys, words):
    run(monkeypatch, *words)
    assert capsys.readouterr().out == HELP
```

- **Fixtures**: `monkeypatch` and `capsys` are two of pytest's built-in fixtures. A test names them as arguments, and pytest passes them in. Our own fixtures come with the model
- `monkeypatch` sets `sys.argv` as if `babykev help` had been typed, and reverts it after the test
- `capsys` catches what the program printed, so the test can compare it with `HELP`
- **`@pytest.mark.parametrize`** runs the function once for each value of `words`: one function, four tests. `just test -k help_is_printed -v` lists them, `[words0]` to `[words3]`

Two tests fail, and the tests are right

Kind: unit, both: one function each.

```
def test_score_confidence_is_zero_for_a_guess():
    assert score_confidence([1 / 3] * 3) == pytest.approx(0)
```

A pure guess over three levels reported confidence 0.5. A guess should have confidence 0. Confidence runs from 1, when the model is sure, down to 0, when it knows nothing. A model that gives every level the same chance knows nothing. pytest: `assert 0.5 == 0 ± 1.0e-12`

```
$ sed -n '82,86p' src/babykev/api.py
$ just test -k score_confidence_is_zero
```

```
def test_rounded_probabilities_still_sum_to_one():
    keys = [f"intent{i}" for i in range(77)]
    answer = to_answers([[1 / 77] * 77], [{"id": "intent", "type": "choice", "keys": keys}])[
        "intent"
    ]
    assert sum(answer["probabilities"].values()) == pytest.approx(1, abs=0.02)
```

77 equally likely options, each rounded to two decimals, summed to 0.77. Probabilities sum to 1. pytest: `assert 0.77 == 1 ± 0.02`

```
$ sed -n '89,90p;99p' src/babykev/api.py
$ just test -k rounded_probabilities
```

kev found and fixed both, days later. A test written on the first day finds on the first day what a user would have found later.

First, the two fixes

The two tests that failed in the last step pass now. The tests did not change. The code did.

- **A guess now has confidence 0.** When the model cannot tell the levels apart, the answer used to say it was half sure. Now it says it is not sure at all. The code measured how spread out the answer was against the wrong yardstick: the whole length of the scale, instead of how spread out a guess is. With three levels, each given a third:
  - How spread out the answer is: the levels are 0, 1 and 2 steps from the first, so ⅓×0 + ⅓×1 + ⅓×2 = **1 step**
  - Old yardstick, first level to last: **2 steps**. Confidence 1 – 1/2 = **0.5**
  - New yardstick, how spread out a guess is, around the middle level: ⅓×1 + ⅓×0 + ⅓×1 = **0.67 steps**. 1 – 1/0.67 = **–0.5**. The formula counts anything below 0 as 0, so an answer as spread out as a guess, or more, has confidence **0**
- **The probabilities add up to 1 again.** Each one was rounded to two decimal places. With 77 options, each 0.013 became 0.01, and the total came to 0.77. Now they are rounded to four places, and the total stays close to 1

This step also brings the tools. We run them next, and the next commit holds what they left behind.

ruff format: style without arguments

- Python had several formatters, each with many options. Go shipped one, `gofmt`, with none, and Python followed with Black
- The ruff formatter copies Black's style and is much faster. Almost no options, so nobody argues about quotes or line breaks again
- `ruff format` is what you call. We map this to `just fmt`. If we had used Black, we would do the same.
- `just fmt` rewrites the files. `just fmt --check` only says which it would change
- On `api.py` as it arrived: 114 lines became 141, and the longest line went from 209 characters to 107
- The formatter changes how the code looks, never what it does: after `just fmt`, `just test` still says 52 passed

ruff can also act as a linter

- A linter is a tool that analyzes code for errors, style issues, and compliance with coding standards. It reports; a formatter changes the file
- A formatter leaves a wrong line wrong, only tidier. A linter catches some wrong code: `F821` is a name used but never defined. `ruff rule F821` explains it
- `ruff check` tells you some of these:

Rules	Find
`E`, `W`, `F`	Errors, and names nobody uses
`I`	Imports out of order
`B`	Likely bugs: a mutable default, a `zip` that silently stops early
`UP`, `SIM`	Old syntax, and code that could be simpler
`S`	Security: `eval`, a hard-coded password
`N`	Names: classes in CamelCase, functions and variables in lower case
`ANN`, `D1`	Missing type annotations and docstrings

Every rule is in `pyproject.toml`, with a comment saying why.

ruff can fix some of these

One can start with `ruff check --fix`. This is mapped to `just lint --fix`.
For our code:

After	Findings
the commit	79
`just fmt` (the formatter)	70
`just lint --fix`	69
`just lint --fix --unsafe-fixes`	37

- `ruff rule CODE` explains any code
- The last 37 need a person: a docstring, a type, a name. That is step 03

`uv` Groups: sequester packages only developers need

```toml
[project]
dependencies = ["pydantic", "fastcore", "torch", "transformers", "peft", "safetensors"]

[dependency-groups]          # for working on the project, never shipped
test = ["pytest", "pytest-cov", "hypothesis"]
lint = ["ruff", "ty", "pre-commit", "yamllint", "validate-pyproject", "hadolint-bin"]
dev  = [{include-group = "test"}, {include-group = "lint"}]
```

`uv sync` gives a developer everything. `uv sync --no-dev` gives an image only what the program runs (step 05b).

A person (agent) needs to remove the remaining errors

There were 36 lint errors the tools could not fix:

- we needed one-line docstrings on every class and function
- there were missing types on test arguments, on returns
- names a reader can say: `K` and `L` became `k` and `n`
- one more test, for a score with a single level

Result: 53 tests, coverage 100%, 0 findings.

What `pre-commit install` writes

`just setup` runs `pre-commit install`, which writes one file, `.git/hooks/pre-commit`:

```
#!/usr/bin/env bash
# File generated by pre-commit: https://pre-commit.com
INSTALL_PYTHON=/Users/rahul/Projects/babykev-replay/.venv/bin/python3
ARGS=(hook-impl --config=.pre-commit-config.yaml --hook-type=pre-commit)
...
if [ -x "$INSTALL_PYTHON" ]; then
    exec "$INSTALL_PYTHON" -mpre_commit "${ARGS[@]}"
```

- Before every commit, git looks in `.git/hooks/` for a file named `pre-commit`, and runs it if it is there
- This file only hands over: it starts pre-commit, from this folder's `.venv`, and tells it which config to read
- The path is this machine's, so the file is never committed. Every clone runs `just setup` to write its own
- Any program in that place would do. Two lines, `#!/bin/sh` and `just check`, would work. pre-commit adds the config file, the hooks it downloads, and the install

`just check`: everything a commit must pass

```
just fmt --check
just lint
just test -q --cov-fail-under=90
just lint-config
```

- Coverage below 90% fails the check
- `just lint-config` checks the files that are not Python: every YAML file, `pyproject.toml`, the lockfile against it, and the justfile's own format

`just lint-config`: the files that are not Python

Four checks, one for each kind of file:

Check	The file it reads	New package?
`yamllint --strict .`	every YAML file, such as `.pre-commit-config.yaml`	yes
`validate-pyproject pyproject.toml`	`pyproject.toml`	yes
`uv lock --check`	`uv.lock`, against `pyproject.toml`	no, uv
`just --fmt --check --unstable`	the `justfile`	no, just

`just --show lint-config` prints the recipe:

```
# Lint the config files: the YAML files, pyproject.toml, the lockfile and this justfile
[group('quality')]
[no-exit-message]
lint-config *args:
    #!/usr/bin/env sh
    set -e
    if [ "$*" = help ]; then just help lint-config; exit 0; fi
    uv run yamllint --strict .
    uv run validate-pyproject pyproject.toml
    uv lock --check
    just --fmt --check --unstable
```

`set -e` stops at the first check that fails.

yamllint: checks every YAML file

```yaml
# .yamllint, its settings
extends: default
ignore-from-file: .gitignore
rules:
  line-length: {max: 100}
  document-start: disable  # a file need not start with a line of three dashes
```

- `uv run yamllint --strict .` reads every YAML file in the repository. When all is well it prints nothing
- `--strict` makes a warning count as a failure, not only an error
- One line of `.pre-commit-config.yaml` indented two spaces too far: `10:15 error syntax error: mapping values are not allowed here (syntax)`. Line 10, column 15
- A broken hook config would break every commit. yamllint sees it first

validate-pyproject: checks `pyproject.toml`

- `pyproject.toml` has an agreed format, written down as a standard: which sections there are, which keys each may hold, and what kind of value each key takes
- `uv run validate-pyproject pyproject.toml` checks the file against it. When all is well: `Valid file: pyproject.toml`
- One key misspelt, `requires_python` instead of `requires-python`:

```
`project` must not contain {'requires_python'} properties
Invalid file: pyproject.toml
```

`uv lock --check`: is the lockfile up to date?

- No new package: uv is already here. `--check` only compares `uv.lock` with `pyproject.toml` and changes nothing
- When they agree: `Resolved 29 packages`
- Raise pydantic's lowest allowed version in `pyproject.toml` from 2.11 to 2.12, and do not run `uv lock`:

```
error: The lockfile at `uv.lock` needs to be updated, but `--check` was provided.

hint: To update the lockfile, run `uv lock`.
```

`just --fmt --check`: is the justfile formatted?

- No new package: just has its own formatter. `--unstable` is needed because just still marks the formatter as unstable
- When the justfile is formatted, it prints nothing
- Write `program:="babykev"` with no spaces, and it prints the justfile as a diff, 70 lines. Two of them carry the change, and the last says why it failed:

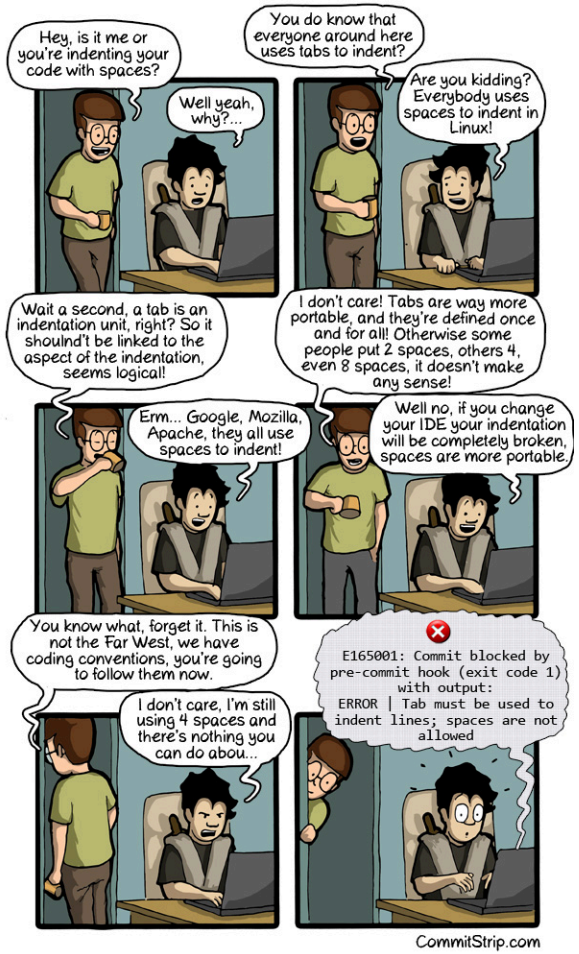
```
-program:="babykev"
+program := "babykev"
...
error: Formatted justfile differs from original.
```

pre-commit: the checks run themselves

- Running every check by hand before each commit gets tiring, and gets forgotten
- A git hook runs before every commit, and refuses the commit if a check fails
- **The hook runs `just check`.** The hook and your terminal run the same thing
- Two more hooks refuse a file over 1 MB and a `.env` file
- `just setup` installs the hook, so nobody has to remember

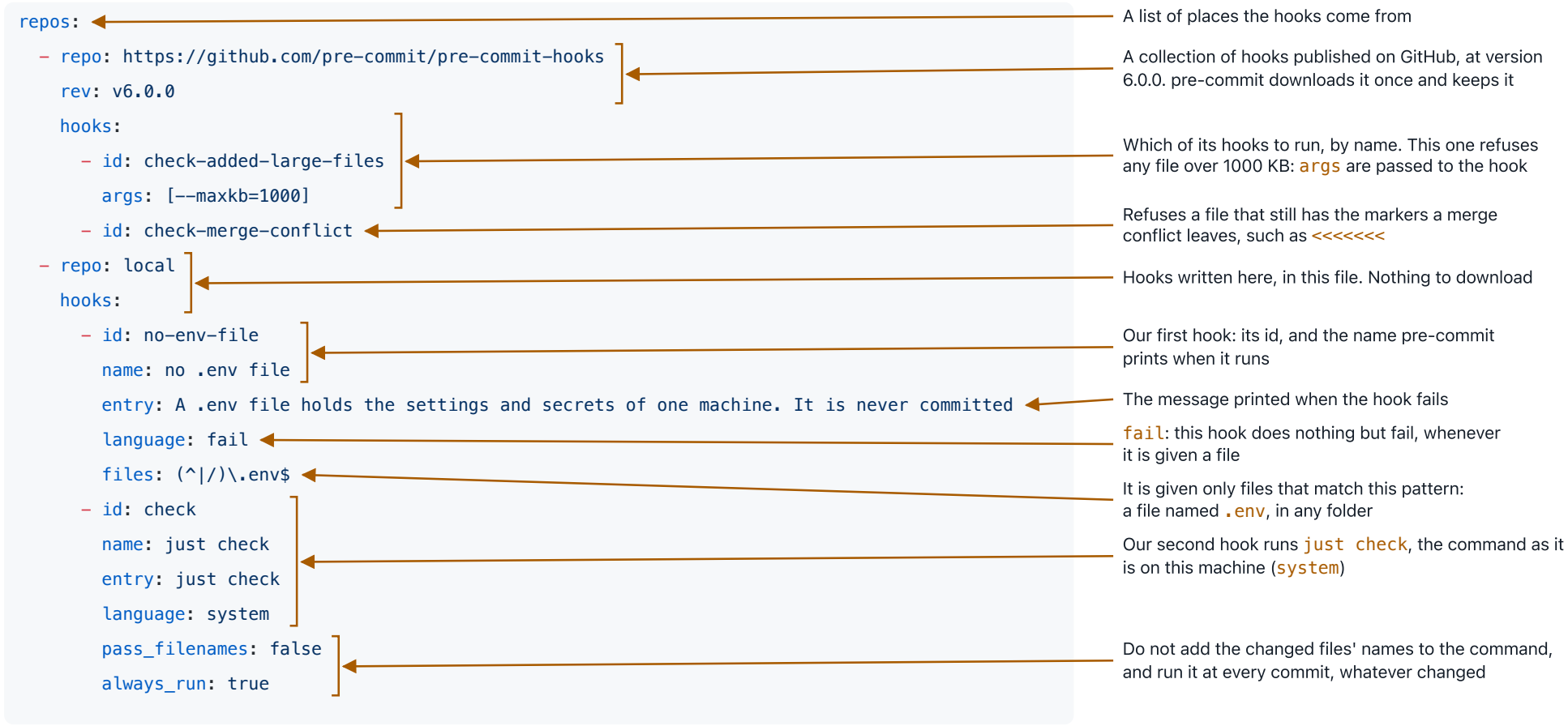
A commit that fails the check does not happen.

The indentation question



CommitStrip, *The Indentation Question*, 14 February 2013, commitstrip.com

The hook's config, line by line



Other moments git can run a hook

A hook is a script git runs at a moment of its own. It lives in `.git/hooks/`, which is not part of the repository, so every clone installs its own: `just setup` does it for us.

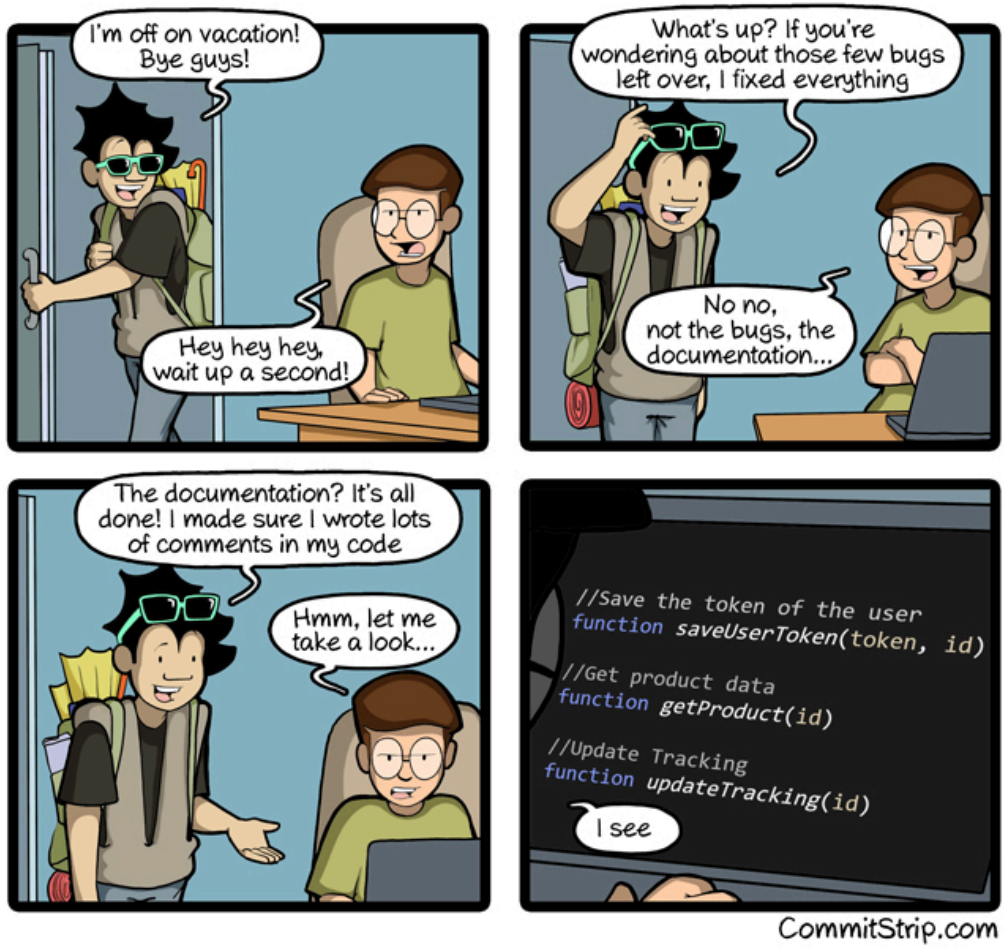
Hook	When git runs it	What it is for
`pre-commit`	before a commit is made	refuse what must not go in. **Ours**
`commit-msg`	on the message, before the commit is made	require a shape for the message
`pre-push`	before a push	checks too slow to run on every commit
`post-checkout`, `post-merge`	after moving to another commit	follow the move, for example update the environment
`prepare-commit-msg`, `pre-rebase`, `post-commit`	around a commit	templates, guards, notices

A hook is like a callback in a training loop: code that runs at a set moment. These hooks run on your machine; a server such as GitHub can run hooks of its own when you push, called server-side hooks.

We use one, `pre-commit`. The others wait until something needs them.

pre-commit is one tool among several that install hooks: husky (for JavaScript projects), lefthook, overcommit. Or none: a script you write yourself in `.git/hooks/`, or a folder of scripts in the repository that `git config core.hooksPath` points git to.

Documentation, just before a holiday



CommitStrip, *Documentation just before vacation*, 27 July 2016, commitstrip.com

The program documents itself

Command	Prints
`babykev docs list`	every module, class and function, one line each
`babykev docs module api`	one module as plain Markdown
`babykev docs check`	what is typed and documented, and every gap. Exit 1 if any

- Plain Markdown, no links: for people at a terminal, and for agents
- fastcore's `docments` reads the comments beside the parameters

Types and docs, written once

```
def line_of(
    sym: Code,  # A function, method or class
) -> int:  # The line its definition starts on
    """Find the line a symbol's definition starts on, to keep symbols in source order."""
```

- The **type** is in the signature
- The **meaning** is in a comment beside each parameter: a **docment**
- The **docstring** says what the function is for
- Nothing is written twice, so nothing can disagree

The gaps

`just docs check` holds every function to one rule: every parameter typed and with a comment, the return typed and with a comment unless it is `None`, a docstring, and a description on every model field.

File	Typed and documented
`babykev.docs`	26 of 26
`babykev.cli`	1 of 1
`babykev.api`	0 of 12
`test_api.py`	23 of 29
`test_cli.py`	0 of 4
`test_docs.py`	0 of 16

- 50 of 88, and 84 gaps: 62 parameters and 10 returns with no comment, 11 model fields with no description, 1 method with no docstring
- Only `babykev.docs`, the documentation tool itself, came with its comments written. The rest get theirs at step 04b
- `just check` does not run `docs check` yet. If it did, the hook would refuse this commit, because `docs check` fails here

The site is built from the same text

- `docs/apidocs.py` writes a reference page per module from `babykev docs`, before every render
- Quarto renders the site into `docs/_site/`. `just docs` builds it
- A name in backticks becomes a link to its entry

Docments and the formatter

- The formatter never joins a line that carries a comment, so docments survive `just fmt`
- One way to lose one: a line longer than 100 characters. The formatter wraps it, and the comment lands away from its parameter
- Only `docs check` notices. So lines stay at 100

Step 04b: every function documented

- 84 gaps filled: 72 docments, 11 field descriptions, 1 docstring. 18 of the docments are in `api.py`, 54 in the tests
- `docs check`: 88 of 88
- `just docs check` joins `just check`. From now on, the hook refuses a commit with any gap

What documented code looks like

```
def render(
    v: JSONContent,  # Text, a number, a list or an object, from a request
    indent: int = 0,  # How deeply nested this value is; each level indents by two spaces
) -> str:  # The text the model sees
```

- The type says `int`; the docment says what the number means. A docment is for what the type cannot say

```
class Noul(BaseModel):
    type: Literal["noul"] = Field(description="Marks the question as yes-or-no")
```

- A pydantic model has no signature to put a comment on, so each field carries `description=`. `docs check` reads it the way it reads a docment

`model.py`: four ideas

- **Pack** the state and every question into one sequence
- **Mask** it, so each question sees the state and itself, never another question
- **Restart the positions**, so every question sits right after the state
- **Point**: score each option against the question's decide token

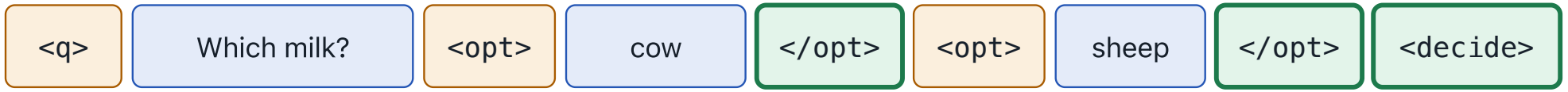
One forward pass answers every question, as if each had been asked alone.

Packing: one sequence for everything

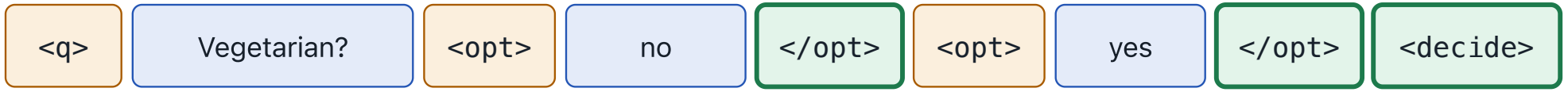
Segment 0: the state — positions 0 … S−1



Segment 1: question 1 — positions S, S+1, …



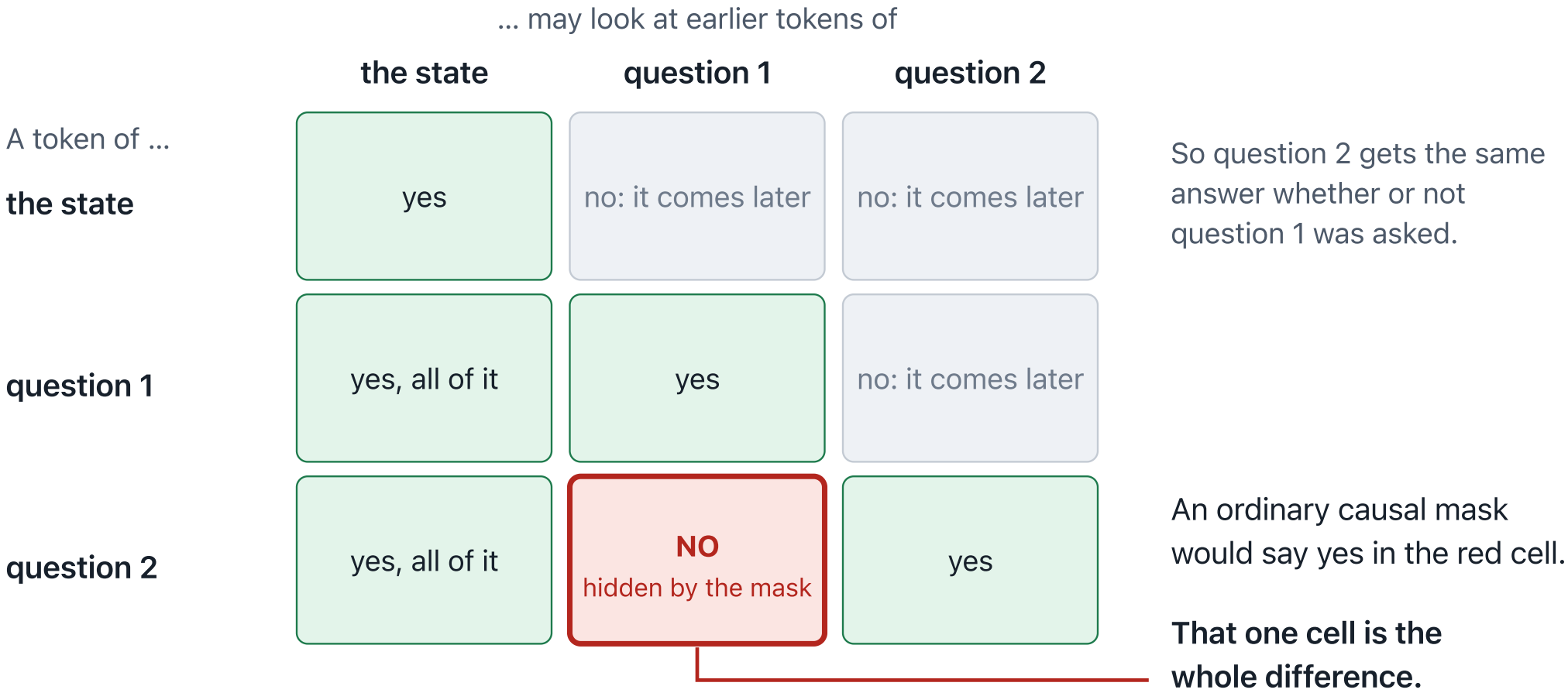
Segment 2: question 2 — positions S, S+1, … again



a delimiter token · the caller's text · a delimiter the answer is read from

Five delimiter tokens mark the parts. A caller's text can never produce one.

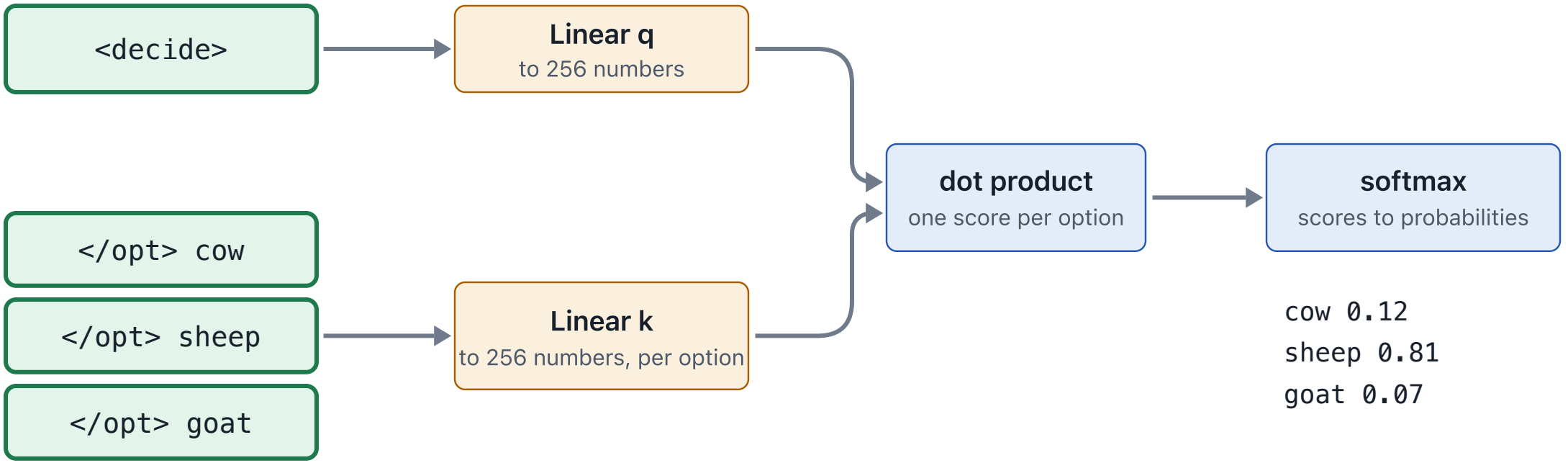
The mask, as designed: questions cannot see each other




```
attend(i, j)  if  j <= i  and  (seg[j] == 0  or  seg[j] == seg[i])
```

The pointer head: compare, do not classify

What the model computed at ...



The options are compared with the question. There is no fixed list of classes: any number of options, any text.

What is trained

Part	Parameters	Trained?
Qwen2.5-0.5B, the language model	about 494 million	No. Frozen
The LoRA adapter, rank 16	about 8.8 million	Yes
The pointer head	about 0.5 million	Yes

The smoke test we will soon see prints it: 503M parameters, 9.3M of them trainable.

Fixtures: what a test needs, passed to it

A test needs things to work on before it can check anything: a sample request, a tokenizer, a model. Not the real ones: small stand-ins, made up for the tests. A **fixture** makes one of them, in one place, for every test that asks.

```
@pytest.fixture
def record() -> dict[str, Any]:          # stand-in data: one request, three questions
    return {"state": "Roquefort. A blue cheese from France, ...", "questions": [...]}

@pytest.fixture(scope="session")
def tok() -> PreTrainedTokenizerFast:     # a stand-in tokenizer, one token per character
    return tiny_tokenizer()

@pytest.fixture
def model(tok: PreTrainedTokenizerFast) -> DecisionModel:   # a stand-in model, built on tok
    return DecisionModel("tiny", "cpu", backbone=tiny_backbone(len(tok))).eval()
```

- A test names the fixture as an argument; pytest calls it and passes in what it returns
- `tok` is built once for the whole run, `scope="session"`. `record` and `model` are built fresh for every test, so no test can spoil them for the next
- A fixture can ask for another: `model` takes `tok`
- `capsys` and `monkeypatch` at step 01 were pytest's own fixtures. These are ours

Writing a fixture

At the beginning we used a helper. Six tests call it by hand:

```
def cheese_request():
    return SystemOneRequest.model_validate({...})

def test_request_holds_one_model_per_question_type():
    questions = cheese_request().questions
```

Now, a fixture. A test asks for it by name:

```
@pytest.fixture
def record() -> dict[str, Any]:
    return {"state": "Roquefort. A blue cheese from France, made from sheep's milk.", "questions": [...]}

def test_positions_restart_after_the_state(tok, record): ...
```

- To write one: a function that returns what the test needs, marked `@pytest.fixture`, in `conftest.py` if more than one file needs it
- If the test leaves something to clean up, `yield` it instead of returning it. The code after `yield` runs when the test is done

`conftest.py`: fixtures for every test file

- pytest reads `tests/conftest.py` before any test. Every test file under `tests/` can use its fixtures without importing anything
- Ours contains three fixtures: `tok`, the tiny tokenizer; `model`, a tiny model; `record`, one request with a state and three questions
- 8 test functions ask for them: all 8 for `tok`, 4 for `record`, 2 for `model`. pytest runs them as 12 tests: one of the 8 runs once for each of the five delimiters
- `just test --fixtures` lists every fixture pytest can see: its own first, then ours, under `fixtures defined from conftest`, each with its docstring

Fixtures in use

Three tests, each asking for its own mix of the three fixtures (types left out):

```
def test_positions_restart_after_the_state(tok, record):                # Kind: unit
    enc = encode(tok, record)
    ...

def test_probabilities_sum_to_one_for_every_question(model, tok, record):  # Kind: integration
    probs = model.probs(encode(tok, record))
    assert [len(p) for p in probs] == [2, 3, 3]

def test_smoke_reports_on_every_question_of_the_sample(model, tok):        # Kind: integration
    report = list(smoke(model, tok, read_lines()))
    ...
```

- `[2, 3, 3]`: the `record` fixture's three questions have 2, 3 and 3 options. A fixture's data is known in advance, so a test can expect exact numbers
- Each test gets its own fresh `model` and `record`; all of them share one `tok`
- `just test --fixtures-per-test -k "probabilities_sum or positions_restart"` shows which fixtures each of the two tests uses, and where each is defined

The tiny model: a test that uses it

Kind: integration: packing, the model and the head together, on the tiny model. `tests/unit/test_model.py`, lines 137 to 146:

```python
def test_probabilities_sum_to_one_for_every_question(
    model: DecisionModel,  # The tiny model
    tok: PreTrainedTokenizerFast,  # Its tokenizer
    record: dict[str, Any],  # A record with three questions
) -> None:
    """Each question gets one probability per option, and they sum to 1."""
    probs = model.probs(encode(tok, record))
    assert [len(p) for p in probs] == [2, 3, 3]
    for p in probs:
        assert p.sum().item() == pytest.approx(1)
```

- The test never says "tiny". It names `model` and `tok` as arguments, and pytest passes it the tiny model and the tiny tokenizer
- `encode(tok, record)` turns the record into tokens with the tiny tokenizer. `model.probs(...)` runs those tokens through the tiny model and the pointer head
- The code that runs is our real `DecisionModel`, from `model.py`. Only the backbone and the tokenizer under it are small
- Nothing is trained, so the probabilities mean nothing. The test checks what is true of any probabilities: one per option, and they add up to 1

The tiny model: where `model` and `tok` come from

`tests/conftest.py`, lines 16 to 27, contains two fixtures:

```
@pytest.fixture(scope="session")
def tok() -> PreTrainedTokenizerFast:
    return tiny_tokenizer()

@pytest.fixture
def model(tok: PreTrainedTokenizerFast) -> DecisionModel:
    return DecisionModel("tiny", "cpu", backbone=tiny_backbone(len(tok))).eval()
```

`tests/tiny.py` contains two ordinary functions, with no `@pytest.fixture` on them:

```
def tiny_tokenizer() -> PreTrainedTokenizerFast: ...                # line 16
def tiny_backbone(vocab_size: int, seed: int = 0) -> Qwen2Model: ...  # line 31
```

- **The fixtures**, `tok` and `model`: pytest calls them and passes what they return to every test that names them. `tok` is built once for the whole run (`scope="session"`). `model` is built again for each test (the default scope)
- **The ordinary functions**, `tiny_tokenizer` and `tiny_backbone`: they build the two small parts. Any code can call them, as the two fixtures do
- **Why the builders are not fixtures**: our `model` fixture takes no arguments and always returns one model, without an adapter. The adapter test needs a tiny model with an adapter, so it calls the function itself, at line 163: `DecisionModel("tiny", "cpu", lora=4, backbone=tiny_backbone(len(tok)))`
- `test_model.py` also calls `tiny_tokenizer()` at line 14, `TOK = tiny_tokenizer()`, for two tests further down (lines 82 and 115)

The tiny model: what the two functions build

Both are built in memory, with no download:

```
chars = sorted(set(string.printable))
vocab = {"[UNK]": 0, **{c: i + 1 for i, c in enumerate(chars)}}
```

```
config = Qwen2Config(vocab_size=vocab_size, hidden_size=32, intermediate_size=64,
                     num_hidden_layers=2, num_attention_heads=2, num_key_value_heads=2, ...)
return Qwen2Model(config)
```

- **The tokenizer**: one token for each of the 100 printable characters, one for each of the five delimiters, and `[UNK]`, the token for any other character. 106 tokens in all. "Roquefort" is 9 tokens, one per letter
- **The backbone**: Qwen's own code, `Qwen2Model`, built smaller: 2 layers of 32 numbers, where the real model has 24 layers of 896
- Its weights are random, drawn from a fixed seed, so they are the same at every run. 24,224 parameters, against the real backbone's 494 million

The tiny model: why it is enough

- What we test is exact: which token may see which, where the positions start, which parameters get a gradient. That is true for any weights, trained or random
- What it cannot tell: whether the answers are any good, how the real tokenizer splits words, how fast it runs on a GPU
- What it buys: no download, no GPU, a few seconds
- It is a working, simplified stand-in for the real model. Testers call such a stand-in a fake

Hermetic: only our code can make these tests fail. No network and no download, and they run on the CPU, which every machine has. So they run on every commit.

Kinds of tests

Kind	What it runs	Our example	Is our example hermetic?
Unit	one function, alone	`test_score_confidence_is_zero_for_a_guess`: one function, given three numbers	yes
Integration	several of our parts together, with stand-ins for what comes from outside	`test_probabilities_sum_to_one_for_every_question`: packing, the model and the head. `test_smoke_reports_on_every_question_of_the_sample`: each request of the sample file through the contract, the packing, the model and the answers. Both on the tiny model	yes: the tiny model stands in for the real one
End-to-end, or system	the whole program with its real parts, the way a user runs it	`just smoke local`: the whole smoke, on the real model	no: it needs the download and a device
Smoke	a quick end-to-end run: does it start, and answer at all?	`just smoke local` again: our end-to-end run is a smoke	no
Acceptance	does it meet a promise to its users, such as an answer within 50 ms?	none yet	

- An end-to-end test uses the real parts, so something outside our code can make it fail: it is never hermetic
- An integration test is hermetic only when what comes from outside is replaced. Ours replace the real model with the tiny one; one that talks to a real database is not hermetic
- The tiny model is not a kind of test. It is the stand-in for the real model, and the `model` fixture returns it to the tests that name it
- A test's name says what it calls, not what kind of test it is: `test_smoke_reports_on_every_question_of_the_sample` calls our function `smoke`, but it is an integration test, not a smoke test
- Teams draw the lines between these names differently. When you call a test an integration test, say what runs in it
- From here on, each test we read names its kind first

What a smoke test is

- The name comes from electronics: switch a new circuit on, and see whether it smokes
- It asks: does the whole thing start, and give an answer at all? Not: is the answer right
- Fast and shallow. Run it first, before the slow and deep tests
- Ours: load the model, ask the 15 sample questions, print each answer beside its label

The smoke: everything, on the real model

Kind: smoke, which is end-to-end: the real model. Run by hand, not in `just test`.

`just smoke local` loads Qwen2.5-0.5B with a fresh adapter and an untrained head, asks the 15 sample questions, and prints each answer beside its label.

- On this Mac's GPU: 6 of 15 right
- Nothing is trained, so these are guesses. What the run proves is that the pieces fit
- Every check is green

ty: a type checker

- A type says what kind of value something is: a whole number, a list of numbers, a tokenizer
- Since step 03, every function of ours says the types it takes and the type it returns, we use this in docments
- ty compares every call and every `return` with what the function says. It does this by reading the code, without running it
- `ty check` runs it on the whole project. The recipe is `just types`
- Here we will we run ty by hand and read what it finds. `just check` does not include it yet, so a commit with negative ty findings still goes through
- At the next step `just check` runs ty too. From then on, a commit with a negative ty finding is refused

currently we get six findings

`just types` prints six findings, each with a file, a line and a code:

Where	Code	What ty says
`model.py:68`	`invalid-return-type`	`load_tokenizer` says it returns a tokenizer. `AutoTokenizer.from_pretrained`, which it returns, says it may return `None`
`model.py:97`	`not-iterable`	the result of `convert_tokens_to_ids` may be a single number, which cannot be split into five names
`test_model.py:49`	`not-iterable`	the same
`test_model.py:100`	`not-subscriptable`	the same result, followed by `[4]`: a single number has no item 4
`test_model.py:111`	`invalid-argument-type`	the same result, given to `set(...)`, which needs a list or something like it
`test_model.py:120`	`not-iterable`	the same as line 49

The first comes from one call. The other five come from one other call, `convert_tokens_to_ids`.

The five findings from one call

`model.py`, line 97:

```
s_id, q_id, o_id, c_id, d_id = tok.convert_tokens_to_ids(SPECIAL)
```

- `SPECIAL` is the list of the five delimiters. `convert_tokens_to_ids` turns tokens into their numbers
- The library says it returns `int | list[int]`: one number when given one token, a list of numbers when given a list
- We give it a list of five, so it returns a list of five numbers, and the line de-structures them into five names. The code runs, and the tests pass
- ty reads only what the library says, not what happens at run time. "It may be one number" is enough: one number cannot be split into five names
- In the next step we will call this once, in a new function `delimiter_ids`, which always returns a list. It also makes `load_tokenizer` stop with an error when it gets no tokenizer. Then ty finds nothing, and `just types` joins `just check`

The problem with the mask as written.

This is what the mask should be, token by token: say we have a small request of six tokens: two for the state, two for question 1, two for question 2. They are numbered 1 to 6, in the order they are packed:

Token	1	2	3	4	5	6
Part	state	state	question 1	question 1	question 2	question 2

The mask says, for each token, which tokens it may look at. Each row below is one token. Along its row, `#` means "may look at this token", `.` means "may not":

```
                  may look at token:  1 2 3 4 5 6
token 1, state                        # . . . . .
token 2, state                        # # . . . .
token 3, question 1                   # # # . . .
token 4, question 1                   # # # # . .
token 5, question 2                   # # . . # .
token 6, question 2                   # # . . # #
```

- No token may look at a token after it: everything right of the diagonal is `.`
- The tokens of question 2, rows 5 and 6, may not look at question 1, tokens 3 and 4. This is the red cell of the earlier picture, token by token
- So question 2 gets the same answer whether or not question 1 was asked

All green, and the mask is wrong

Here there is no test that checks the mask. Every check in `just check` passes: 92 tests, 94% coverage, 139 of 139 documented. Coverage counts `branch_mask` as tested, because its lines ran. No test looks at the mask the code here builds.

The code at step 05 builds a different mask from the last slide. For the same six tokens:

```
                      should be      step 05 builds
token 1, state        # . . . . .    # . . . . .
token 2, state        # # . . . .    # # . . . .
token 3, question 1   # # # . . .    # # # . . .
token 4, question 1   # # # # . .    # # # # . .
token 5, question 2   # # . . # .    # # # # # .
token 6, question 2   # # . . # #    # # # # # #
```

Rows 5 and 6 differ: at step 05, question 2 may look at question 1's tokens, 3 and 4.

```
$ sed -n '140p' src/babykev/model.py
```

```
same = torch.ones_like(causal) | (s[None, :] == 0)
```

`same` should be yes only when two tokens are in the same question, or the other one is in the state. `torch.ones_like(causal)` makes it yes for every pair, so every token may look at every token before it.

A check that sees it

At step 05 there is no test of the mask. A plain Python function can still check it. It draws the right mask for the six tokens of the grid next to the mask `branch_mask` builds, and marks each row:

```
from babykev.model import branch_mask

def check_small_request():
    picture = ["#.....", "##....", "###...", "####..", "##..#.", "##..##"]
    mask = branch_mask([0, 0, 1, 1, 2, 2], "cpu")[0, 0] == 0
    rows = ["".join("#" if ok else "." for ok in row) for row in mask]
    for want, got in zip(picture, rows, strict=True):
        print(want, got, "ok" if want == got else "WRONG")
```

```
#..... #..... ok
##.... ##.... ok
###... ###... ok
####.. ####.. ok
##..#. #####. WRONG
##..## ###### WRONG
```

Rows 5 and 6 are WRONG: question 2 looks at question 1. This is not a test: nothing in `just check` runs it.

Normally we would fix before we commit.

But say Pavlos distracted us, so we comitted and went to his class :-)



We had two problems left over from step 05

1. **A type problem.** `just types` finds six problems. Five come from one line, `model.py` line 97: `tok.convert_tokens_to_ids(SPECIAL)`, which gives back a list of five numbers that the line splits into five names. ty reads only that the library may give back one number, and one number cannot be split. The sixth is `load_tokenizer`, which may give back `None`
2. **A mask problem.** `branch_mask` lets question 2 look at question 1 (`model.py` line 140). Every check passes, because there is no test of the function that builds the mask

Step 05a deals with both.

The tiny model again: how a test gets it

Step 05 built a tiny tokenizer and a tiny model in memory, in `tests/tiny.py`: 106 tokens, 2 layers of 32 numbers, 24,224 parameters. A test gets them as fixtures. `tests/conftest.py`, lines 16 to 27:

```
@pytest.fixture(scope="session")
def tok() -> PreTrainedTokenizerFast:
    return tiny_tokenizer()

@pytest.fixture
def model(tok: PreTrainedTokenizerFast) -> DecisionModel:
    return DecisionModel("tiny", "cpu", backbone=tiny_backbone(len(tok))).eval()
```

- A test builds nothing. It names `model` and `tok` as arguments, and pytest passes in what these two functions return. The tests below do that
- `tok` is built once for the whole run. `model` is built fresh for each test
- A third fixture, `record`, lines 30 to 40, is one request about Roquefort with three questions: is it a blue cheese, which milk, how firm

```
$ sed -n '16,27p' tests/conftest.py
```

Test properties, not examples

- **Isolation**: change question 2, and question 1's answer must not move
- **Packed equals separate**: three questions in one pass give the numbers of three passes
- **Order does not matter**
- **The mask's rule**, for any request: no token sees a later one, no question sees another

They are true for any weights, so the tiny model is enough.

```
$ just test
```

A picture checks one size

At the end of step 05, five lines of Python drew the right mask for six tokens, and rows 5 and 6 were WRONG. Suppose someone fixes the mask like this instead, so that only the question right before is blocked:

```
same = (s[None, :] == s[:, None]) | (s[None, :] == 0) | (s[None, :] < s[:, None] - 1)
```

The same five lines, run again, print `ok` six times. The picture has only two questions, so it cannot tell.

A request of a different size: one state token, then three questions of one token each, parts `[0, 1, 2, 3]`. It needs its own picture, drawn by hand. On the left the picture, on the right what the wrong fix builds:

```
#...   #...   ok
##..   ##..   ok
#.#.   #.#.   ok
#..#   ##.#   WRONG
```

Row 4: question 3 looks at question 1.

- We have no function that draws the right mask for any size. A request can have 1 to 6 state tokens, then 0 to 4 questions of 1 to 5 tokens each: 4,686 shapes, and each would need its own picture
- So we do not test with pictures. We state the rule once, and let the computer try shapes

hypothesis: state the rule, the computer tries the shapes

`hypothesis` is a Python library for tests. We say what a request may look like, and it draws requests for us. `test_mask.py`, line 10:

```
shapes = st.tuples(st.integers(1, 6), st.lists(st.integers(1, 5), max_size=4))
```

That says: a state of 1 to 6 tokens, then a list of up to 4 questions of 1 to 5 tokens each.

```
@given(shapes)
def test_no_question_sees_another_question(shape): ...
```

- `@given(shapes)` runs the test again and again, each time with a shape drawn at random from the 4,686. By default, up to 100 times
- If a shape breaks the rule, hypothesis shrinks it: it tries smaller shapes until none smaller breaks, and shows that one
- We write the rule once. hypothesis looks for the shape we did not think of
- hypothesis allows each run 200 ms. The first run that uses torch can take longer on a slow or busy machine, and the test would then fail by chance. `tests/conftest.py` turns the deadline off, and says why in a comment

```
$ sed -n '10p;58,59p' tests/unit/test_mask.py
$ sed -n '43,51p' tests/conftest.py
```

hypothesis finds the size for us

Kind: unit: `branch_mask` alone. We write the wrong fix of the last slide into `branch_mask`, and read the rule, written as a test:

```
$ perl -pi -e 's/^    same = \(s\[None, :\] == s\[:, None\]\) \| \(s\[None, :\] == 0\)$/    same = (s[None, :] == s[:, None]) | (s[None, :] == 0) | (s[None, :] < s[:, None] - 1)/' src/babykev/model.py
$ sed -n '58,65p' tests/unit/test_mask.py
$ just test -k no_question_sees_another
```

```
@given(shapes)
def test_no_question_sees_another_question(shape):
    """A token of one question never attends to a token of another."""
    seg = torch.tensor(segments(*shape))
    siblings = (seg[:, None] != seg[None, :]) & (seg[:, None] > 0) & (seg[None, :] > 0)
    assert not (allowed(seg.tolist()) & siblings).any()
```

`segments` gives the part of each token: 0 for the state, k for question k. `siblings` is yes for two tokens in two different questions. `allowed` is the mask read as yes or no. The rule: no pair is both.

hypothesis tries shapes until the rule breaks, then shrinks the failure to `shape = (1, [1, 1, 1])`: one state token, three questions of one token each, parts `[0, 1, 2, 3]`. It is the request we drew by hand. We chose that size. hypothesis found it on its own:

```
allowed       siblings
# . . .       . . . .
# # . .       . . # #
# . # .       . # . #
# # . #       . # # .
```

Row 4, column 2 is yes in both: token 4, in question 3, may look at token 2, in question 1. The assert fails.

```
$ git restore src/babykev/model.py
```

How we tested isolation

Kind: integration: packing, the model and the head, on the tiny model.
We put step 05's mask line back first. `test_model.py`, lines 145 to 157:

```
$ perl -pi -e 's/^    same = \(s\[None, :\] == s\[:, None\]\) \| \(s\[None, :\] == 0\)$/    same = torch.ones_like(causal) | (s[None, :] == 0)/' src/babykev/model.py
$ sed -n '145,157p' tests/unit/test_model.py
$ just test -k a_question_cannot_see_its_sibling
$ git restore src/babykev/model.py
```

```python
def test_a_question_cannot_see_its_sibling(model, tok, record):
    before = logits_of(model, tok, record)
    changed = {**record, "questions": list(record["questions"])}
    changed["questions"][1] = {"instr": "Where from?", "options": ["Spain", "Italy"], "label": 0}
    after = logits_of(model, tok, changed)
    assert torch.equal(before[0], after[0])
    assert torch.equal(before[2], after[2])
    assert before[1].shape != after[1].shape
```

- `model`, `tok` and `record` are the fixtures of `conftest.py`: the tiny model, its tokenizer, and a request with three questions
- `logits_of` packs the request and gives, for each question, one score for each of its options
- We score the request, rewrite question 2, and score again. Questions 1 and 3 must get exactly the same scores. Question 2 must get a different number of scores: 3 options became 2
- With step 05's mask, the line for question 3 fails: before `[-0.2301, -0.2331, -0.2340]`, after `[-0.2266, -0.2302, -0.2312]`. Question 3 could look at question 2, so rewriting question 2 moved its scores. The line for question 1 passes: question 1 comes first and cannot look at what follows

The tests see it, and the one-line fix

We put step 05's mask line back, and run all the tests:

```
$ perl -pi -e 's/^    same = \(s\[None, :\] == s\[:, None\]\) \| \(s\[None, :\] == 0\)$/    same = torch.ones_like(causal) | (s[None, :] == 0)/' src/babykev/model.py
$ just test
```

8 failed, 95 passed.

- The commit hook, from step 03, runs `just check`, and `just check` runs the tests. So this commit would now be refused
- At step 05 the same commit went through, because no test looked at the mask. `just check` refuses only what its tests can see

```
$ git restore src/babykev/model.py
$ sed -n '151p' src/babykev/model.py
```

```
    same = (s[None, :] == s[:, None]) | (s[None, :] == 0)
```

A token may look at the state, or at its own question. Step 05 had `torch.ones_like(causal)` where the first part is.

The smoke run on Qwen, again: 4 of 15

`just smoke local`, on the real model, Qwen2.5-0.5B, with the right mask:

```
$ just smoke local
```

- 6 of 15 with the wrong mask, 4 with the right one
- A pure guess on these fifteen questions expects 4.75 right: five yes-or-no, five of five options, five of four levels
- Untrained, the model guesses. With the wrong mask, some guesses were right for the wrong reason

The type problem: two guards

```
$ sed -n '64,80p' src/babykev/model.py
```

```
tok = AutoTokenizer.from_pretrained(name)
if not isinstance(tok, PreTrainedTokenizerBase):
    raise TypeError(f"{name} has no tokenizer this model can use")
return tok
...
ids = tok.convert_tokens_to_ids(SPECIAL)
return ids if isinstance(ids, list) else [ids]
```

- `load_tokenizer` checks what came back. If it is not a tokenizer, it stops with an error. So it never gives back `None`
- `delimiter_ids` is the one place that calls `convert_tokens_to_ids`. If the library gives one number, it makes a list of one. So it always gives back a list. `encode` and the tests call it, in place of the five calls ty complained about

```
$ just types
```

`just types` prints: All checks passed!

ty joins `just check`

```
$ just --show check
$ just check
```

- `just check` now runs `just types` between `lint` and the tests
- The commit hook runs `just check`. So a commit with a type problem is refused, at ty, before the tests run
- At step 05 we could commit with six ty findings. From now on we cannot

Not everyone has a Mac

- Students run Windows, Linux, WSL, or a VM in the cloud
- The platforms we train on are Linux machines in someone else's building
- An image is a Linux machine described in one file, the `Dockerfile`, and built anywhere Docker runs

How Docker was born



A widely shared meme; its author is unknown. imgflip.com/i/24ac74

Two images from one lockfile

	dev	run
Contains	every dependency with the dev tools, uv, just, Quarto, git	Python and what the program needs
Starts	a shell, with your folder mounted	`babykev`
For	working on the project on any system, and CI	a platform that runs the program
Size	1.84 GB	1.15 GB

The packages are installed first and the code after, so a change to the code does not reinstall torch.

The run image is built in stages, and only the last one ships

```
FROM ghcr.io/astral-sh/uv:0.12.13 AS uv

FROM python:3.13-slim-trixie AS build
COPY --from=uv /uv /usr/local/bin/uv
...
COPY pyproject.toml uv.lock .python-version ./
RUN uv sync --locked --no-dev --no-install-project
COPY src ./src
RUN uv sync --locked --no-dev --no-editable

FROM python:3.13-slim-trixie AS run
COPY --from=build /opt/venv /opt/venv
COPY data ./data
...
ENTRYPOINT ["babykev"]
CMD ["help"]
```

Stage 1, `uv`: uv's own image. Nothing runs in it: the next stage copies the uv program out of it

Stage 2, `build`: a fresh Python where the environment is made. It is not shipped

The packages first, from the lockfile: torch and the rest. This step is reused until the lockfile changes

Then our code, and install it. A change to the code reinstalls one package, not torch

Stage 3, `run`: a fresh Python again. Nothing of stage 2 comes with it, except what is copied next

The finished environment, and only that. No uv, no just, no Quarto, no dev tools, no leftovers of the build

What starts: `babykev`. With no word, `help`

`docker build --target run` stops at the stage named `run`.

The dev image: the same Python, with the tools

```
FROM python:3.13-slim-trixie AS dev
ARG TARGETARCH
ARG JUST=1.50.0
ARG QUARTO=1.8.26
RUN apt-get update \
    && apt-get install --yes --no-install-recommends ca-certificates curl git \
...
RUN curl --fail --silent --show-error --location --output /tmp/quarto.deb \
...
COPY --from=uv /uv /uvx /usr/local/bin/
RUN uv tool install "rust-just==${JUST}" \
    && git config --system --add safe.directory '*'
WORKDIR /work
COPY pyproject.toml uv.lock .python-version ./
RUN UV_COMPILE_BYTECODE=1 uv sync --locked --no-install-project
COPY . .
RUN uv sync --locked
EXPOSE 8765
CMD ["bash"]
```

The dev image: a fresh Python, as the run image starts. It shares nothing with `build` or `run`

git and curl, from Debian

Quarto, from its release: the docs site needs it

uv, copied out of the first stage

just, from PyPI. git is told every folder is safe: a folder mounted from the host belongs to the host's user

The packages first, this time with the dev group: pytest, hypothesis, ruff, ty, hadolint and the rest of `just check`

Then the whole project, and install it

The port the step browser uses

It starts a shell, not `babykev`

`docker build --target dev` needs only the first stage, `uv`, and none of `build` or `run`.

`just image`

```
just image build dev        the dev image, named by the commit
just image shell            a new container of it: then just check, as on the Mac
just image start            or one kept running; shell goes into it, stop ends it
just image run smoke        the smoke in the run image
```

- An image is a file on disk. A container is a running copy of it
- `just` runs on your machine, and it runs `docker`. In the run image there is no `just`: the container runs `babykev`, so `just image run smoke` ends up as `babykev smoke` in the container
- The dev image has `just`. `just image shell` gives a shell in it, and there every recipe works, as on the Mac: `just check` is green, `just smoke local` gives 4 of 15 on the CPU, and `git` works in the mounted folder
- In the run image, on the CPU: 4 of 15, the same fifteen answers as on the Mac's GPU, line for line. Eight times slower

hadolint: a linter for Dockerfiles

It reads the `Dockerfile` and names the rules it breaks. On our file it says nothing. It found two things in the first draft.

1. Two `RUN` lines in a row. Lines 68 and 69 are one `RUN`: the `just` install and the `git` setting, joined with `&&`. Split them in two, and hadolint says:

```
$ uv run hadolint Dockerfile
Dockerfile:69 DL3059 info: Multiple consecutive `RUN` instructions. Consider consolidation.
```

We fixed it: one `RUN`.

2. Debian packages without a pinned version. Delete the line that declines the rule, and hadolint says:

```
$ uv run hadolint Dockerfile
Dockerfile:48 DL3008 warning: Pin versions in apt get install. Instead of `apt-get install <package>` use `apt-get install <package>=<version>`
```

We declined it, and said why beside the line, lines 46 to 48.

- A linter's rule is a default, not a law
- It is a line of `just lint-config`, so the hook runs it

What should not go in the hook?

Along the way we added a check at almost every step: format, lint, types, the tests, the docs, the config files. `just check` runs all of them, and the commit hook runs `just check`. So should everything go in?

- A check that needs the network, a download, a GPU or a lot of time can fail for reasons outside our code
- A hook that fails for those reasons cannot be trusted, and people learn to skip it
- So the question for every check: can anything but our code make it fail? Only the checks where the answer is no belong in the hook

The next slides sort our tests by that question.

Kinds of tests, and which can run on every commit

Kind	What runs	Ours	Is ours hermetic?
Unit	one function	a confidence function on a list of numbers	yes
Integration	several of our parts together, with stand-ins for what comes from outside	packing, the mask, the model and the head; and the whole smoke. Both on the tiny model	yes: the tiny model stands in for the real one
End-to-end, or system	the whole program with its real parts, as a user runs it	the smoke on the real model, from the Hub: `test_real_model.py`	no
Smoke	a quick end-to-end run: does it start and answer at all?	the same smoke on the real model, by hand: `just smoke local`	no
Acceptance	does it meet a requirement, such as an answer within 50 ms	none yet	

- An end-to-end test uses the real parts, so something outside our code can make it fail: it is never hermetic. An integration test is hermetic only when what comes from outside is replaced
- pytest has no built-in kinds. You name your own markers in `pyproject.toml`, put `@pytest.mark.NAME` on a test, and choose with `-m NAME`
- Our one marker is `system`. It is on the one test that is end-to-end: the smoke on the real model

2 unit tests, 0 integration tests



Made popular by Randal Olson, January 2016; the clip's author is unknown.
giphy.com/gifs/2-unit-tests-0-integration-3o7rbGT9rrEJa4Y5LG

Hermetic, by experiment

The smoke on the real model is now a test too, marked `system`. `just test` leaves it out.

The mask and isolation tests of step 05a are integration tests by kind, and they have no marker: they are hermetic, so they run at every commit. The marker `system` is only on the one test that is not hermetic: the smoke on the real model, which is a system test.

With `--network none`	Result
`just check`	passes
`just test -m system`, the model in the cache	passes
the same, with an empty cache	fails: it cannot reach the Hub

What still passes with the network pulled can run on every commit.

A test is anything that can fail and stop you

More generally, a test is anything that can fail and stop a commit. All of these are tests in that sense:

What is checked	How	Recipe	On every commit?
Functions, the mask, the model	pytest, hypothesis, the tiny model	`just test`	yes
Types	ty	`just types`	yes
Documentation	`babykev docs check`	`just docs check`	yes
Format and lint	ruff	`just fmt --check`, `just lint`	yes
Config files, the Dockerfile	yamllint, validate-pyproject, uv, hadolint	`just lint-config`	yes
The real model, from the Hub	the smoke, marked `system`	`just test -m system`	no: not hermetic

Most of these are not pytest. All of them are `just`.

You may even call it a gate



Distracted girlfriend, a widely shared stock photo (imgflip.com/memetemplate/118429510). The labels are ours

The docs, published

- The site holds the reference pages of each module and, since step 05a, a Guide: the model in four ideas, and the mask as a grid in a notebook
- `just docs publish` builds the site and pushes it to the `gh-pages` branch
- GitHub Pages serves it: rahuldave.github.io/babykev
- The built site never enters `main`